

Full Stack Viva Study Guide

A straightforward, conversational guide to explaining your project's technology stack.

1. Python (The Backend Brain)

Python handles the heavy lifting, logic, and database connections in the SmartBank backend.

Data Structures

- **Lists vs Tuples:** Lists `[1, 2]` can be changed (mutable). Tuples `(1, 2)` cannot be changed (immutable), making them faster and safer for fixed data.
- **Dictionaries:** `{key: value}`. Works like a real dictionary. Best for fast data lookups. Keys must be immutable.
- **Sets:** `{1, 2}`. Only stores unique items. Great for instantly removing duplicates.

Decorators (@ syntax)

A decorator is a function that wraps around another function to change its behavior. Example: In Flask, we use `@app.route('/login')` to tell Python 'run this function when the user visits the login URL'. We also use `@login\_required` to check security before running a page.

Generators & yield

If you load 1 million users using `return`, it crashes your computer's memory. `yield` turns a function into a Generator. It gives you one user at a time, pauses, and waits for you to ask for the next one, saving massive amounts of RAM.

2. HTML (The Skeleton)

HTML creates the structure of the webpage. It is not a programming language; it is a markup language.

The DOM (Document Object Model)

When the browser reads your HTML, it creates a family tree of objects called the DOM. JavaScript talks to this DOM to change what the user sees without reloading the page.

Semantic HTML

Instead of using generic `

` tags everywhere, we use tags that have meaning, like ``, ``, and ``. This helps search engines (SEO) and screen readers for blind users understand the page structure.

3. CSS (The Paint & Layout)

CSS makes the HTML look good. It handles colors, spacing, and responsive design for mobile.

The Box Model

Every element on a webpage is a box. The Box Model consists of: Content (the actual text/image), Padding (space inside the border), Border (the line around it), and Margin (space outside the border pushing other elements away).

Flexbox vs Grid

- **Flexbox:** Best for aligning items in a single row or column (like a navigation bar).
- **CSS Grid:** Best for complex 2D layouts with both rows and columns (like a photo gallery or dashboard layout).

Classes vs IDs

Classes (`.btn`) can be used on multiple elements. IDs (`#header`) must be completely unique on the page.

4. JavaScript (The Muscle/Behavior)

JS makes the page interactive. It runs in the user's browser, handling clicks, animations, and fetching data from the Python backend.

Variables: let vs const vs var

- **let:** A variable that can be changed later. Only exists inside the block `{}` where it was created.
- **const:** A variable that cannot be reassigned. Use this by default to prevent bugs.
- **var:** The old way. Avoid using it because it 'leaks' out of blocks and causes weird bugs.

Asynchronous JavaScript (Promises & Async/Await)

When JS asks Python for database info, it takes time. Instead of freezing the whole browser while waiting, JS uses `async/await`. It basically says: 'Send the request, go do other things, and come back to this exact spot when the data arrives.'

Event Listeners

Code that 'listens' for user actions. Example: `button.addEventListener('click', doSomething)`. When the button is clicked, it runs the function.

5. Databases (The Vault)

Databases store persistent data. Without them, everything deletes when you turn off the server.

Relational (SQL) vs NoSQL

- **SQL (like PostgreSQL/SQLite):** Stores data in strict tables with rows and columns. Best when data is highly connected (e.g., Users have Accounts, Accounts have Transactions).
- **NoSQL (like MongoDB):** Stores data as flexible JSON documents. Good for rapidly changing, unstructured data.

Keys & Relationships

- **Primary Key:** A unique ID for every row in a table (like an Account Number).
- **Foreign Key:** A column in one table that links to the Primary Key of another table. This is how you link a Transaction to a specific User.

ACID Properties

Crucial for banking apps! ACID ensures transactions are safe.

- **Atomicity:** 'All or nothing'. If money leaves Account A but fails to reach Account B, the entire transfer is canceled.
- **Consistency:** The database rules are never broken.
- **Isolation:** Two people transferring money simultaneously won't mess up the math.
- **Durability:** Once a transaction is saved, a power outage won't erase it.